

Practical: 3

Implementation of max-heap sort algorithm

Aim: To Implement the max-heap sort algorithm

Algorithm:

Max-Heap Algorithm.

Input: Unsorted Array
Output: Sorted Array

step1: start
step2: Read the Number of elements size
step3: Read the array elements $arr[1]$ to $arr[size]$
step4: call $heapsort(arr, size)$
step5: [Build max-heap] Repeat for $i = size$ down to 1
 • call $MaxHeapify(arr, size, i)$
step6: [Extraction phase] Repeat for $i = size$ down to 2
 • swap $arr[i]$ and $arr[1]$
 • call $MaxHeapify(arr, i-1, 1)$
step7: Display sorted array $arr[i]$ to $arr[size]$
step8: STOP

Sub-Algorithm : $MaxHeapify(arr, size, i)$

step1: set target = i
step2: set $l = 2 * i$
step3: set $r = 2 * i + 1$
step4: If $l \leq size$ and $arr[l] > arr[target]$, set target = l
step5: If $r \leq size$ and $arr[r] > arr[target]$, set target = r
step6: if target $\neq i$:
 • swap $arr[i]$ and $arr[target]$
 • call $MaxHeapify(arr, size, target)$ recursively
step7: Return.

Program:

```
#include <iostream>
using namespace std;

class HeapSort
{
public:
    int arr[50], size;

    void inputArray()
    {
        cout << "Enter size of array: ";
        cin >> size;

        cout << "Enter " << size << " elements: ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void MaxHeapify(int arr[], int size, int i)
    {
        int largest = i;
        int l = (2 * i) + 1;
        int r = (2 * i) + 2;

        if(l < size && arr[l] > arr[largest]){
            largest = l;
        }

        if(r < size && arr[r] > arr[largest]){
            largest = r;
        }

        if(largest != i)
        {
            swap(arr[largest], arr[i]);
            MaxHeapify(arr, size, largest);
        }
    }

    void heapSort()
    {
        for(int i = size/2 - 1; i >= 0; i--)
        {

```

```
        MaxHeapify(arr, size, i);
    }

    for(int i = size - 1; i >=1; i--)
    {
        swap(arr[0], arr[i]);
        MaxHeapify(arr, i, 0);
    }
}

void printArray()
{
    cout << "Sorted array: ";
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

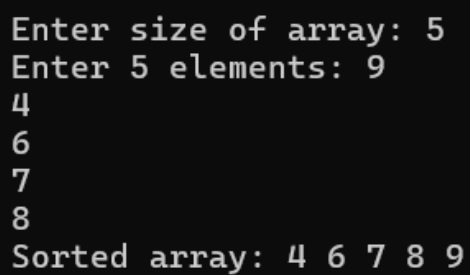
};

int main()
{
    HeapSort obj;

    obj.inputArray();
    obj.heapSort();
    obj.printArray();

    return 0;
}
```

Output:



```
Enter size of array: 5
Enter 5 elements: 9
4
6
7
8
Sorted array: 4 6 7 8 9
-----
```

Time Complexity:

Time-complexity:

```

for(int i = n-1; i >= 1; i--) {
    swap(A[0], A[i]);
    maxHeapify(A, i, 0);
}
    
```

Bestcase: Even if the array already sorted heap sort()

$\Rightarrow$ for(int i = size; i >= 2; i--)

And calls Max-Heapify() after every Execution

Therefore it is sorting phase, still $O(n \log n)$

Bestcase: $O(N \log N)$

Average case:

Build-Heap $O(N)$

Extraction $(n-1) O(\log N)$

Therefore $T(n) = O(N) + (n-1)O(\log N)$

$T(n) = O(N \log N)$

Average case = $O(N \log N)$

Worstcase:

For Each Extraction $O(\log N)$

There are approximately n extractions

$T(n) = n \log n$

Therefore

Worstcase = $O(N \log N)$

Conclusion:

Hence, the Max-Heap Sort algorithm was successfully implemented and executed. It was observed that Heap Sort gives a time complexity of $O(n \log n)$ in the best, average, and worst cases. It is an efficient sorting technique and works well for sorting large sets of data.

FACULTY OF TECHNOLOGY

Department of AI, ML & DS

01AI0506 | DAA Lab Manual

